



CafeBook

PROJECT FILE LIST & PURPOSE

CafeBook is a full-stack cafe seat booking system. Customers can register, browse seating zones, and reserve a table online. Admins can manage all bookings and tables through a dashboard. The app runs entirely from a single **Start CafeBook.bat** file — no separate setup required.

28	7	19	1
TOTAL SOURCE FILES	FRONTEND PAGES	JAVA FILES	DATABASE

► LAUNCHER

FILE	PURPOSE
Start CafeBook.bat	Double-click to run the entire project. Starts the Spring Boot server and automatically opens http://localhost:8080 in the browser after 18 seconds. No other steps needed.

► FRONTEND — PAGES (FRONTEND/ FOLDER)

FILE	PURPOSE
index.html	Homepage. Displays the hero banner, 4 feature cards, and a "How It Works" 3-step guide. This is the first page visitors see.
auth.html	Combined Sign In and Register page. Two tabs on one page — the user can switch between logging in or creating a new account without loading a different page.
book.html	The main reservation form — "Dine With Us". The customer fills in their name, date, time, number of guests, and seating zone, then clicks Confirm Reservation. A confirmation screen with a booking reference (e.g. CB-00001) appears on success.
my-bookings.html	Shows all the logged-in user's reservations. Each booking shows its seat, date, time, status badge, and a Cancel button for active bookings.
admin.html	Admin dashboard. Has two tabs — <i>Bookings</i> (search, filter, change status of any booking) and <i>Tables</i> (add new tables, mark available/unavailable, delete). Only accessible to users with the ADMIN role.
css/style.css	All visual styling for every page in one file — fonts (Playfair Display & Montserrat), colours, layout grids, buttons, cards, toast notifications, navbar, footer, animations, and mobile responsiveness.

js/api.js

The shared brain of the frontend. Loaded by every HTML page. Handles: all API calls to the server, storing/reading the JWT login token, building the navbar and footer, showing toast messages, and utility functions (bookingRef, formatTime12, statusBadge).

► CONFIGURATION (BACKEND/SRC/MAIN/RESOURCES/)

FILE	PURPOSE
application.properties	The single settings file for the whole backend. Controls: database file location, server port (8080), JWT secret key and expiry time, and where Spring Boot looks for the frontend files.

► JAVA — APP STARTUP (2 FILES)

FILE	PURPOSE
BookingApplication.java	The single entry point of the Java app. Contains one line that launches the entire Spring Boot application.
DataInitializer.java	Runs automatically once when the app first starts. Creates the admin account (admin@cafe.com), 6 seating zones (Window Side, Cosy Corner, Outdoor Patio, etc.), and 12 tables. If data already exists, it does nothing — so it is safe to restart the app.

► JAVA — SECURITY & AUTHENTICATION (CONFIG/ — 2 FILES)

FILE	PURPOSE
JwtFilter.java	Handles all JWT (JSON Web Token) work in one place. createToken(email) — called after login/register to build a signed token. doFilterInternal() — runs on every incoming request, reads the token from the Authorization header, and tells Spring Security which user is making the request.
SecurityConfig.java	Defines the access rules for the whole app. Public (no login needed): homepage, auth page, view tables. Login required: making bookings, viewing your bookings. Admin only: /api/admin/** endpoints.

► JAVA — MODELS / DATABASE TABLES (MODEL/ — 4 FILES)

FILE	PURPOSE
User.java	Represents a registered person. Fields: id, name, email, phone, password (hashed, never stored in plain text), role (ADMIN or GUEST), dietary notes, accessibility flag. Maps to the <i>users</i> table in the database.
Zone.java	Represents a named seating area inside the cafe — e.g. "Window Side", "Outdoor Patio", "Lounge". Each table belongs to a zone. Maps to the <i>zones</i> table.

RestaurantTable.java	Represents a physical table. Fields: table number, minimum and maximum capacity, type (STANDARD / GROUP / PRIVATE / MERGED), available flag, accessible flag, special features, and which zone it belongs to. Maps to the <i>restaurant_tables</i> table.
Booking.java	Represents one reservation. Links a User to a RestaurantTable. Stores booking date, time, party size, special notes, status (PENDING / CONFIRMED / COMPLETED / CANCELLED / NO_SHOW), and when it was created. Maps to the <i>bookings</i> table.

► JAVA — DATA SHAPE (DTO/ — 1 FILE)

FILE	PURPOSE
BookingRequest.java	Defines exactly what data the booking form sends to the server when creating a reservation: tableId, bookingDate, bookingTime, partySize, specialNotes, and depositAmount.

► JAVA — REPOSITORIES / DATABASE QUERIES (REPOSITORY/ — 4 FILES)

FILE	PURPOSE
UserRepository.java	Reads and writes the <i>users</i> table. Key method: findByEmail() — used during login and on every JWT-authenticated request to look up who is making the call.
ZoneRepository.java	Reads and writes the <i>zones</i> table. Used by DataInitializer to check whether zones have already been created before seeding.
TableRepository.java	Reads and writes <i>restaurant_tables</i> . Contains the most important query in the app — findAvailable(date, time, partySize) — which returns only tables that are marked available, have enough seats, and are not already booked at that date and time.
BookingRepository.java	Reads and writes the <i>bookings</i> table. Key method: findByUserOrderByCreatedAtDesc() — returns all bookings for a specific user, newest first, for the My Reservations page.

► JAVA — SERVICES / BUSINESS LOGIC (SERVICE/ — 2 FILES)

FILE	PURPOSE
AuthService.java	register() — saves the new user with a BCrypt-hashed password and returns a JWT token. login() — verifies the password using BCrypt, and if correct returns a JWT token. Both return the token and user details so the browser can save them.
BookingService.java	All booking business logic: creating a new booking (links user + table), fetching a user's booking history, cancelling a booking, getting all bookings (admin), and updating a booking's status (admin).

► JAVA — CONTROLLERS / API ENDPOINTS (CONTROLLER/ — 4 FILES)

FILE & URL	PURPOSE
AuthController.java <i>/api/auth/...</i>	Receives login and register requests from the browser. POST <i>/api/auth/login</i> — sign in. POST <i>/api/auth/register</i> — create account. These are the only endpoints that don't require a JWT token.
TableController.java <i>/api/tables/...</i>	Public table data. GET <i>/api/tables</i> — returns all tables (used to build the zone dropdown on the booking form). GET <i>/api/tables/available</i> — returns tables free for a given date, time, and party size.
BookingController.java <i>/api/bookings/...</i>	Requires login. POST <i>/api/bookings</i> — creates a new reservation. GET <i>/api/bookings/my</i> — returns the logged-in user's booking history. PATCH <i>/api/bookings/{id}/cancel</i> — cancels a specific booking.
AdminController.java <i>/api/admin/...</i>	Requires ADMIN role. Booking management: view all bookings, change any booking's status. Table management: add a new table, toggle a table's availability on/off, delete a table.

► BUILD FILE

FILE	PURPOSE
pom.xml	Maven build configuration. Lists all libraries the project depends on: Spring Boot (web server), Spring Security (authentication), Spring Data JPA (database), H2 (embedded database), and JWT (JWT token creation). Maven reads this file to download and compile everything automatically.

► DATABASE (DATABASE/ FOLDER)

FILE	PURPOSE
database/cafebook.mv.db	The live H2 embedded database file. Stores all users, zones, tables, and bookings permanently. This file is created automatically the first time the app starts. It persists between restarts — data is never lost unless this file is deleted.

backend/data/cafebook.mv.db is an old database from before the folder reorganisation and can be safely deleted.